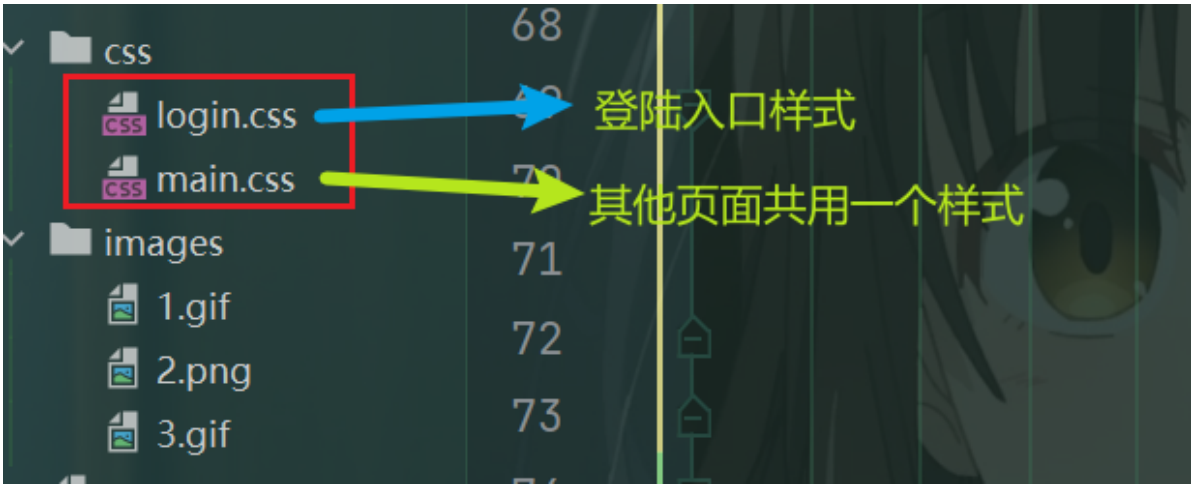
教务管理系统

第一步：写前端

问题1：重复性代码太多

解决方法：

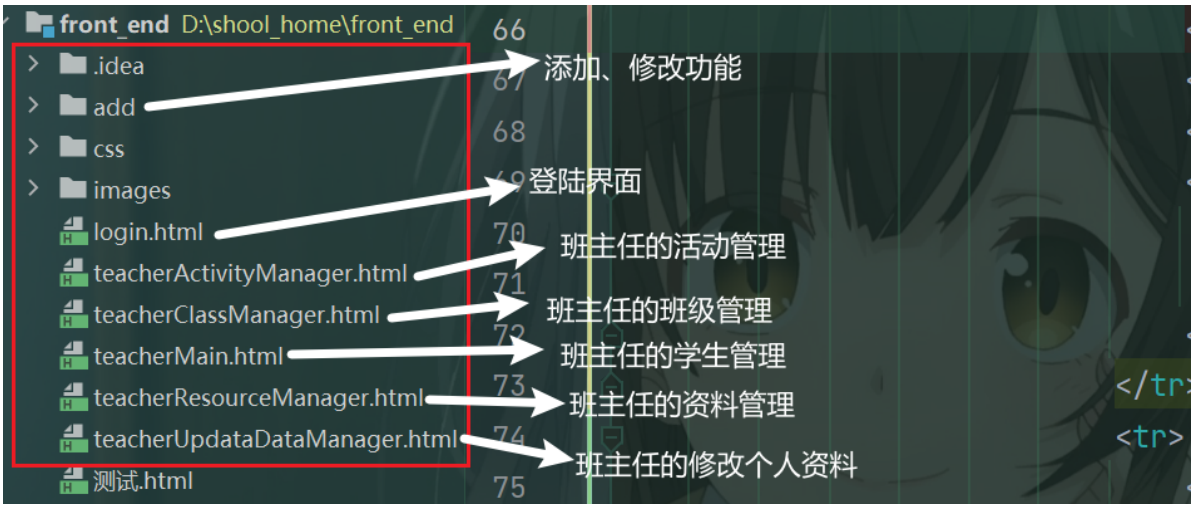
1.css方面



2.html方面

> 写后端的时候再说，把这个问题throw给未来的自己

实现代码



第二步：前后端交互

开发环境

操作系统：Windows

Web 服务器：tomcat9.0.80

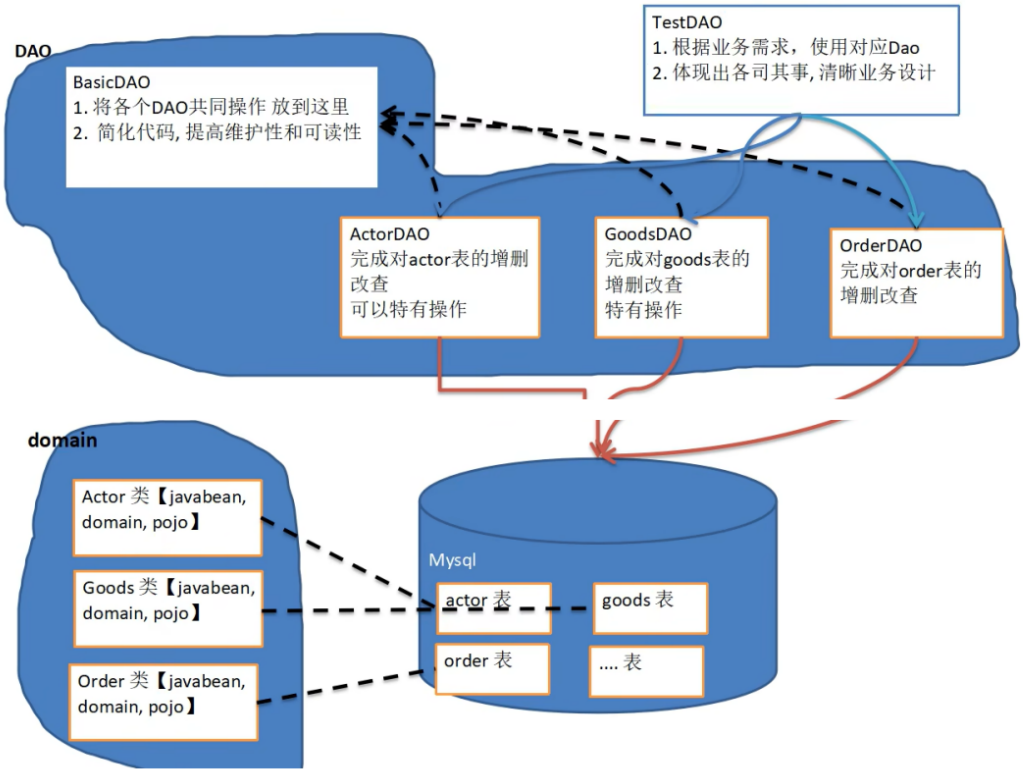
java 开发包：JDK1.8

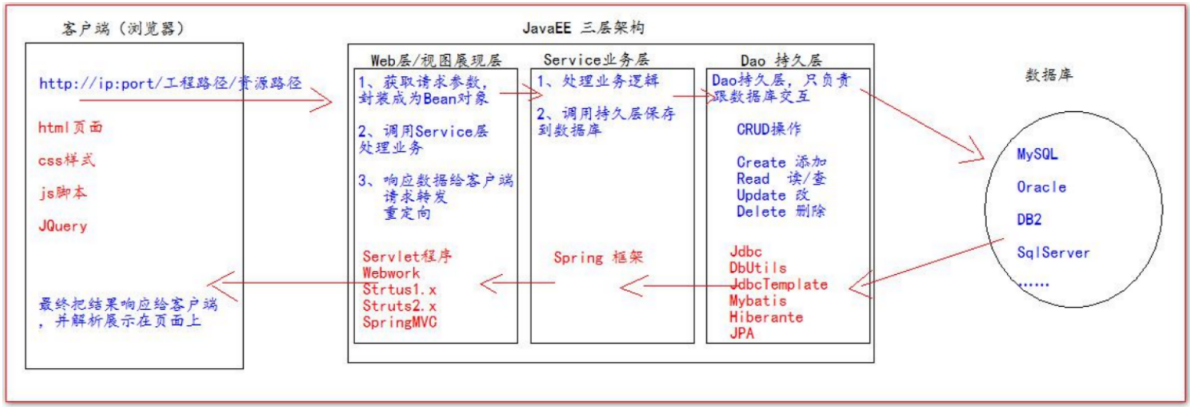
开发工具：IDEA2022.3.3

数据库：MySQL5.7.19

浏览器：Google Chrome

javaEE 项目的三层架构





分层的目的是为了解耦。解耦就是为了降低代码的耦合度。方便项目后期的维护和升级。

web 层	com.atguigu.web/servlet/controller	
service 层	com.atguigu.service	Service 接口包
	com.atguigu.service.impl	Service 接口实现类
dao 持久层	com.atguigu.dao	Dao 接口包
	com.atguigu.dao.impl	Dao 接口实现类
实体 bean 对象	com.atguigu.pojo/entity/domain/bean	JavaBean 类
测试包	com.atguigu.test/junit	
工具类	com.atguigu.utils	

搭建书城项目架构：

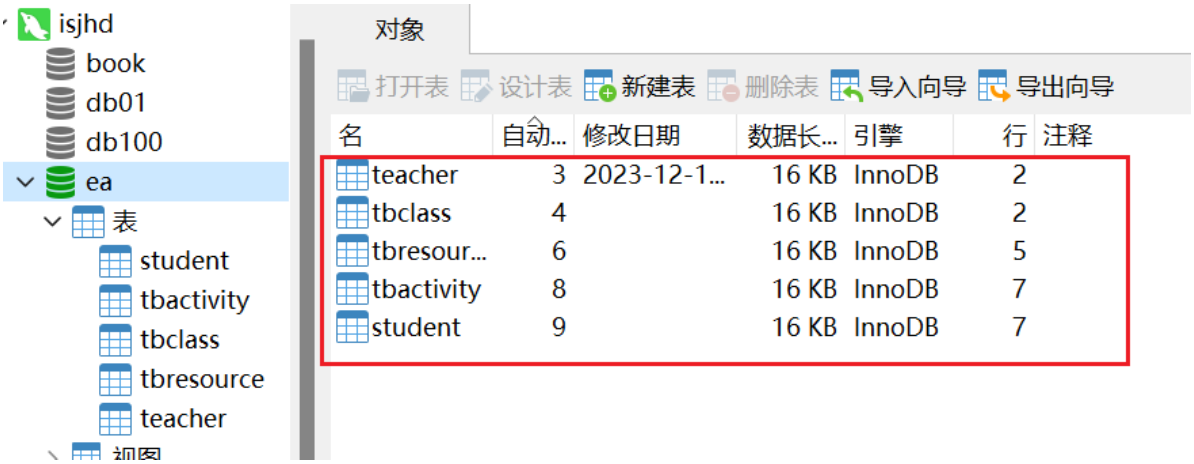



```
|-- pom.xml # Maven 项目管理文件
|-- src
    |-- main # 项目主要代码
    |   |-- java # Java 源代码目录
    |   |   `-- com/example/myapp # 开发者代码主目录
    |   |   |-- controller # 存放 Controller 层代码的目录
    |   |   |-- service # 存放 Service 层代码的目录
    |   |   |-- dao # 存放 DAO 层代码的目录
    |   |   `-- model # 存放数据模型的目录
    |   |-- resources # 资源目录，存放配置文件、静态资源等
    |   |   |-- log4j.properties # 日志配置文件
    |   |   |-- spring-mybatis.xml # Spring Mybatis 配置文件
    |   |   `-- static # 存放静态资源的目录
    |   |       |-- css # 存放 CSS 文件的目录
    |   |       |-- js # 存放 JavaScript 文件的目录
    |   |       `-- images # 存放图片资源的目录
    |   `-- webapp # 存放 WEB 相关配置和资源
    |   |-- WEB-INF # 存放 WEB 应用配置文件
    |   |   |-- web.xml # Web 应用的部署描述文件
    |   |   `-- classes # 存放编译后的 class 文件
    |   `-- index.html # Web 应用入口页面
    `-- test # 项目测试代码
        |-- java # 单元测试目录
```

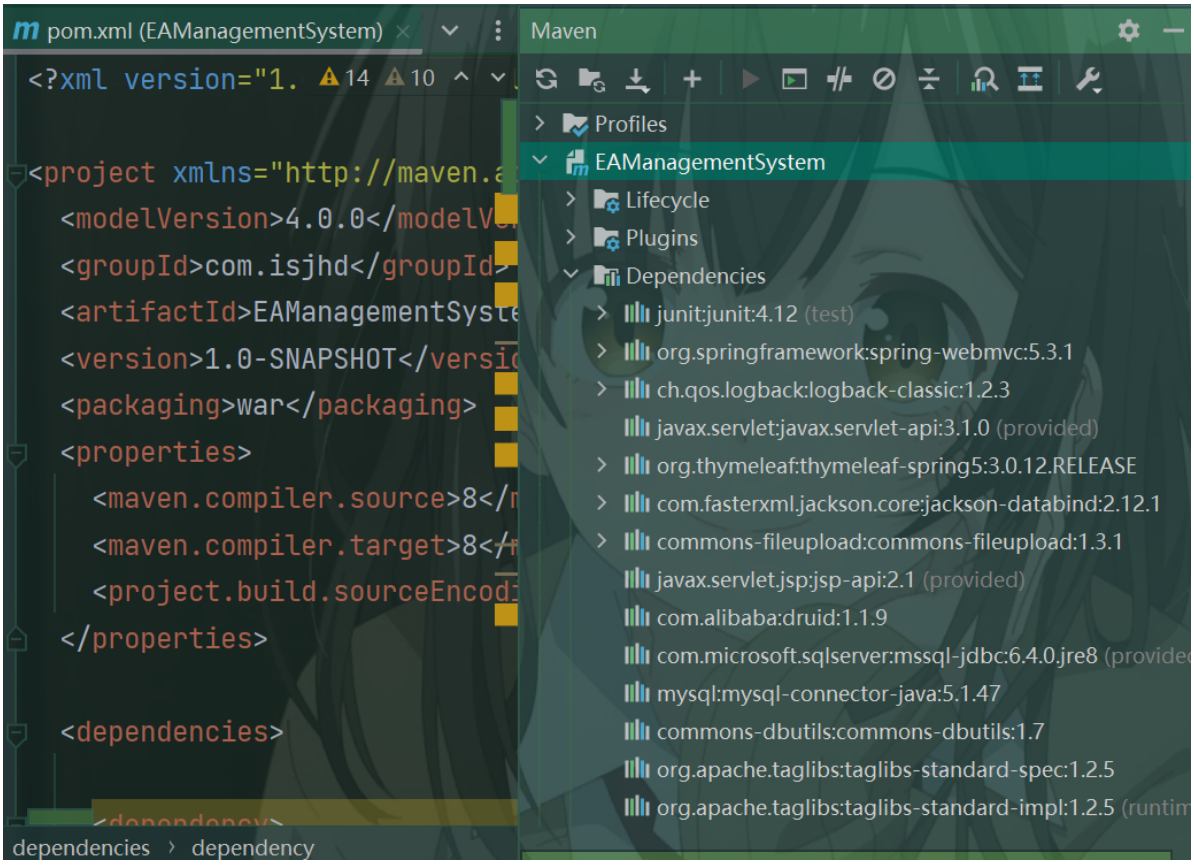
```
        `-- resources # 测试资源目录
```

- pom.xml：Maven 项目管理文件，用于描述项目的依赖和构建配置等信息
- src/main/java：存放项目的 Java 源代码。
- src/main/resources：存放项目的资源文件，如配置文件、静态资源等。
- src/main/webapp/WEB-INF：存放 Web 应用的配置文件。
- src/main/webapp/index.jsp：Web 应用的入口页面。
- src/test/java：存放项目的测试代码。
- src/test/resources：存放测试相关的资源文件，如测试配置文件等。

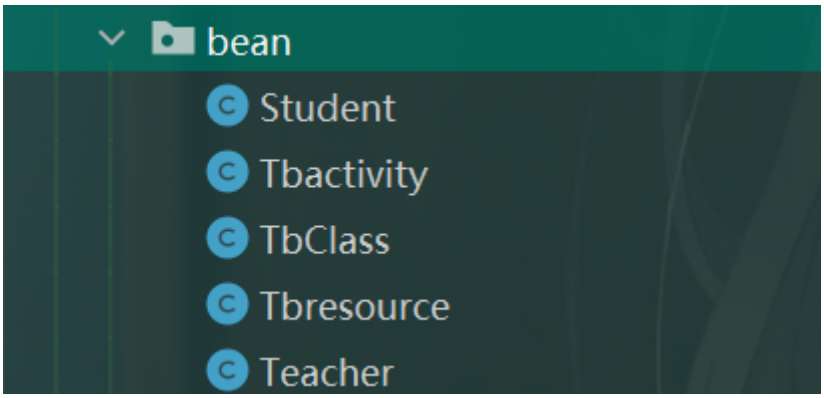
先创建书城需要的数据库和表



导入jar包依赖



编写数据库表对应的 JavaBean 对象



实现数据库和后端的交互

编写 jdbc.properties 属性配置文件

```
#key=value
driverClassName=com.mysql.jdbc.Driver
url=jdbc:mysql://localhost:3306/ea?rewriteBatchedStatements=true
#url=jdbc:mysql://localhost:3306/girls
username=root
password=hsp
#initial connection Size
initialSize=10
#min idle connecton size
minIdle=5
#max active connection size
maxActive=20
#max wait time (5000 mil seconds)
maxWait=5000
```

编写工具类JdbcUtils

```
/* @author i-s-j-h-d
 * @version 1.0
 * 基于druid数据库连接池的工具类
 */
public class JdbcUtils {
    private static DruidDataSource dataSource;
    static {
        try {
            Properties properties = new Properties();
            // 读取 jdbc.properties 属性配置文件
            InputStream inputStream =
JdbcUtils.class.getClassLoader().getResourceAsStream("jdbc.properties");
            // 从流中加载数据
            properties.load(inputStream);
            // 创建数据库连接池
            dataSource = (DruidDataSource)
DruidDataSourceFactory.createDataSource(properties);
        } catch (Exception e) {
```

```
            e.printStackTrace();
        }
    }
}
```

bean实体类

```
public class Student {// 学生表

    private String stuid; //学号
    private String sname; //姓名
    private Integer gender; //性别
    private String phone; //联系方式
    private String address; //家庭地址
    private String clsid; //所在班级名称
    private Integer STATUS; //在籍
}
```

编写父类Dao代码

```
public class BaseDao {
    //使用DbUtils操作数据库
    private QueryRunner queryRunner = new QueryRunner();

    // update() 方法用来执行：Insert\Update\Delete语句
    // return 如果返回-1，说明执行失败，返回其他表示影响到的行数
    public int update(String sql, Object ... args) {
        Connection connection = JdbcUtils.getConnection();
        try {
            return queryRunner.update(connection, sql, args);
        } catch (SQLException e) {
            e.printStackTrace();
            throw new RuntimeException(e);
        }
    }
}
```

编写Dao持久层

```
@Repository
public class StudentDaoImpl extends BaseDao implements StudentDao {
    @Override // 修改学生信息
    public void updateStudent(Student student) {
        String sql = "update student set stuid=?,
sname=?,gender=?,phone=?,address=?, clsid=?,STATUS=? where stuid=?";
        Object[] args =
{student.getStuid(),student.getSname(),student.getGender(),student.getPhone(),

 student.getAddress(),student.getClsid(),student.getSTATUS(),student.getStuid()}
;
        int update = update(sql, args);
        System.out.println(update);
    }
}
```

编写service层

```
@Service
public class StudentServiceImpl implements StudentService {

    private StudentDao studentDao = new StudentDaoImpl();

    @Override// 修改学生信息
    public void updateStudent(Student student) {
        studentDao.updateStudent(student);
    }
}
```

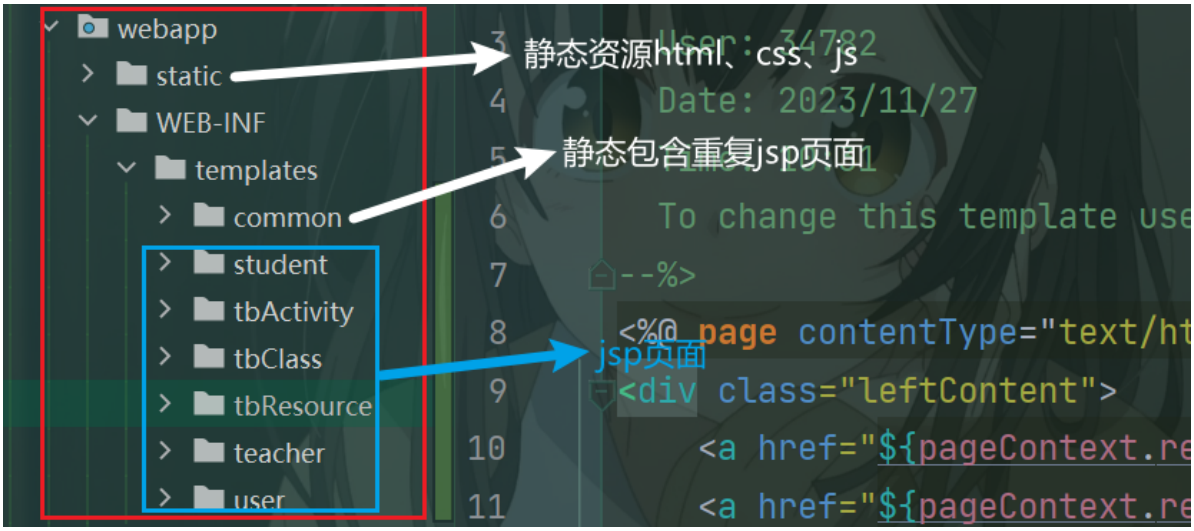
测试

```
public class TestStudent {

    private StudentService studentService = new StudentServiceImpl();

    @Test// 修改学生信息
    public void updateStudent(){
        Student student = new Student("2022090108", "神原骏河", 1, "22244446666",
"日本", "1903", 1);
        studentService.updateStudent(student);
    }
}
```

html页面转换为jsp






```
<%@ page contentType="text/html;charset=UTF-8" language="java" %>
<!--写base标签，永远固定相对路径跳转的结果-->
<%
    String basePath = request.getScheme()
            + "://"
            + request.getServerName()
            + ":"
            + request.getServerPort()
            + request.getContextPath()
            + "/";

    pageContext.setAttribute("basePath", basePath);
%>
<base href="<%=basePath%>">
<link type="text/css" rel="stylesheet" href="static/css/add.css" >
```

处理请求

有需要的处理的请求放在controller的java类中

```java
@RequestMapping("/student")
public String student(){
    return "student/teacherMain";
}

@RequestMapping("/tbClass")
public String tbClass(){
    return "tbClass/teacherClassManager";
}

@RequestMapping("/tbActivity")
public String tbActivity() { return "tbActivity/teacherActivityManager"; }

@RequestMapping("/tbResource")
public String tbResource() { return "tbResource/teacherResourceManager"; }

@RequestMapping("/login")
public String loginJump() {
    return "redirect:/";
}
```

只进行跳转的请求放在springMVC

```xml
<mvc:view-controller path="/modifyStudentsJump" view-name="student/modifyStudents"></mvc:vie
<mvc:view-controller path="/addStudentJump" view-name="student/addStudent"></mvc:view-contro
<mvc:view-controller path="/modifyActivityJump" view-name="tbActivity/modifyActivity"></mvc:
<mvc:view-controller path="/createActivityJump" view-name="tbActivity/createActivity"></mvc:
<mvc:view-controller path="/modifyClassJump" view-name="tbClass/modifyClass"></mvc:view-cont
<mvc:view-controller path="/modifyClassInformationJump" view-name="tbResource/modifyClassInf
<mvc:view-controller path="/uploadClassInformationJump" view-name="tbResource/uploadClassInf
<mvc:view-controller path="/teacherUpdataDataManagerJump" view-name="teacher/teacherUpdataDa
```

实现登录

与数据库交互实现

```java
// 班主任登录
Teacher teacherInput(String username, String password);
```

controller页面代码

```java
@Controller
public class InputController {

    private TeacherService teacherService = new TeacherServiceImpl();

    @RequestMapping("teacherInput")
    public String teacherInput(String username, String password,
                               @RequestParam(value="sex",defaultValue="无")String
sex,
                               HttpServletRequest request){
}
```

jsp页面

```
<h1 class="top">Login</h1>
<div class="input">
    ${ empty requestScope.msg ? "请输入用户名和密码":requestScope.msg }
</div>
<table align="center" border="1" cellpadding="0" cellspacing="0" width="300" h
    <tr>
```

问题

sex.equals("班主任")中sex如果为空，则报错500

```
<td>
    <input type="radio" name="sex" id="administrators" value="管理员"><label for="administrators">管理员</label>
    <input type="radio" name="sex" id="classTeacher" value="班主任"><label for="classTeacher">班主任</label>
</td>
```

解决办法：为sex设置默认值 @RequestParam(value="sex",defaultValue="无")String sex

```
@RequestMapping("/teacherInput")
public String teacherInput(String username, String password,
                           @RequestParam(value="sex",defaultValue="无")String sex,
                           HttpServletRequest request){
    if(sex.equals("班主任")) {
        Teacher teacher = teacherService.teacherInput(username, password);
        if(teacher != null){
```

提示登录失败

```
// 把错误信息，和回显的表单项信息，保存到Request域中
request.setAttribute( s: "msg",  o: "用户或密码错误");
```

```
<div class="input">
    ${ empty requestScope.msg ? "请输入用户名和密码":requestScope.msg }
</div>
```

替换标签内容

```
<tr>
    <th>账户名</th>
    <td><input type="text" name="aname" id="aname" value="${sessionScope.teacher.username}"></td>
</tr>
<tr>
    <th>头像</th>
    <td>
        <input type="file" name="myFile" id="myFile">
        <img src="static/images/${sessionScope.teacher.photo}" align="center" style="...">
    </td>
</tr>
```

问题

```
<th>性别</th>                    性别无法设置value
<td>
    <input type="radio" name="sex" id="nan"><label for="nan">♂</label>
    <input type="radio" name="sex" id="nv"><label for="nv">♀</label>
</td>
```

解决办法：

```
<script type="text/javascript">
   // onload事件是浏览器解析完页面之后就会自动融发的事件
   window.onload = function ()  {
      // document.getELementsByName();是根据指定的name属性查询返回多个标签对象集合
      // 这个集合的操作跟数组一样
      // 集合中每个元素都是dom对象
      // 这个集合中的元套颇序是他们在htmL页面中从上到下的顺序
      var sex = document.getElementsByName("sex");
      var i = 1;
      var j = ${sessionScope.teacher.gender};
      // checked表示复选框的选中状态。如果选中是true，不选中是false
      // checked 这个属性可读，可写
      if(i === j) {
         sex[0].checked = true;
      } else{
         sex[1].checked = true;
      }


   }
</script>
```

实现管理

列表功能的实现

学号	姓名	性别	联系方式	家庭地址	班级	状态	操作
2022090101	忍野忍	♀	22244446666	日本	1903	在读	修改学生信息 开除学生
2022090102	战场原黑仪	♀	22244446666	日本	1903	在读	修改学生信息 开除学生
2022090103	羽川翼	♀	22244446666	日本	1903	在读	修改学生信息 开除学生
2022090104	四宫辉夜	♀	22244446666	日本	1904	在读	修改学生信息 开除学生

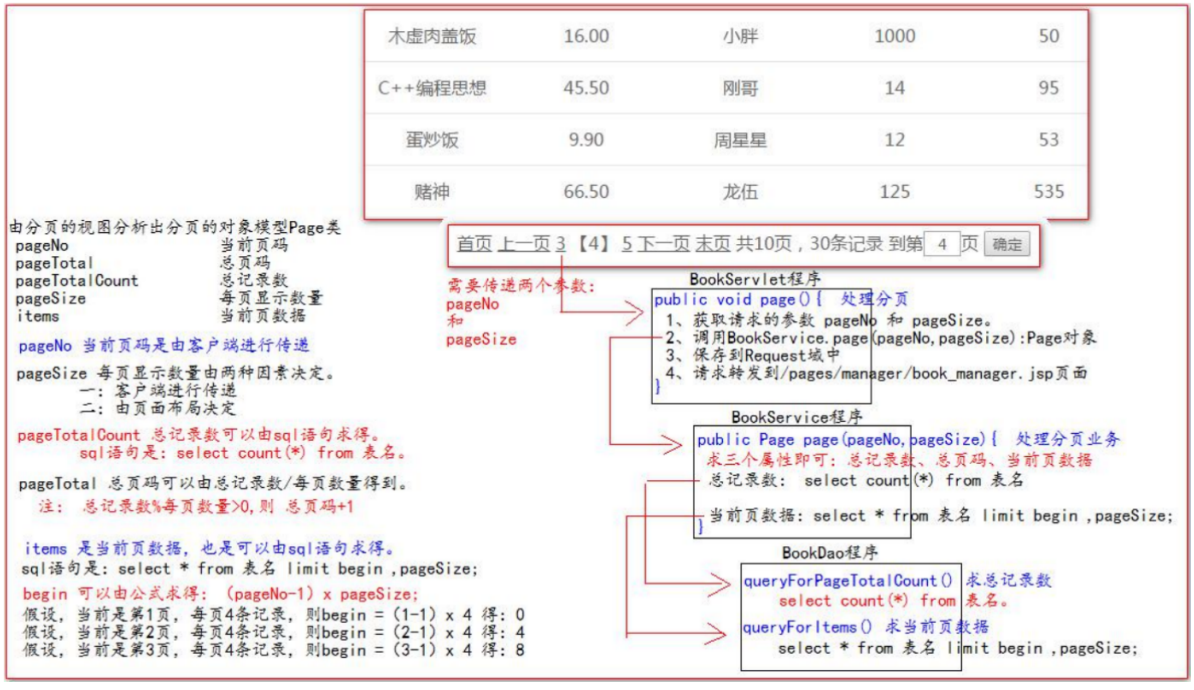
```
<c:forEach items="${requestScope.allStudent}" var="student">
    <tr>
        <td>${student.stuid}</td>
        <td>${student.sname}</td>
        <td><h3>${student.gender==0 ? "♀":"♂"}</h3></td>
        <td>${student.phone}</td>
        <td>${student.address}</td>
        <td>${student.clsid}</td>
        <td>${student.STATUS==1 ? "在读" : "开除"}</td>
        <td>
            <a href="${pageContext.request.contextPath}/modifyStudentsJump">修改学生信息</a>
            <input type="button" value="开除学生">
        </td>
    </tr>
</c:forEach>
```

分页

当前页: 1, 总页数: 1

首页	上一页	下一页	尾页

分页模块的分析



分页模型 Page 的抽取（当前页数，总页数，总记录数， 当前页数据，每页记录数）

```
public class Page<T> {
    // 每页显示页数
    private static final Integer PAGE_SIZE = 2;

    // 当前页码
    private Integer pageNo;

    // 总页码
    private Integer pageTotal;

    // 当前页显示数量
    private Integer pageSize = PAGE_SIZE;

    // 总记录数
    private Integer pageTotalCount;

    // 当前页数据
    private List<T> items;

    public Page() {
    }
}
```

分页的初步实现

DaoImpl 代码:

```
@Override// 总记录数
public Integer queryForPageTotalCount() {
    String sql = "select count(*) from student";
    Number count = (Number) queryForSingleValue(sql);
    return count.intValue();
}

@Override// 当前页数据
public List<Student> queryForPageItems(int begin, int pageSize) {
    String sql = "select * from student limit ?,?";
    return queryForList(Student.class,sql,begin,pageSize);
}
```

ServiceImpl 代码:

```
@Override// 当前页数据
public Page<Student> page(int pageNo, int pageSize) {
    Page<Student> page = new Page<Student>();

    // 设置当前页码
    page.setPageNo(pageNo);
```

```
    // 设置每页显示的数量
    page.setPageSize(pageSize);

    // 求总记录数
    Integer pageTotalCount = studentDao.queryForPageTotalCount();

    // 设置总记录数
    page.setPageTotalCount(pageTotalCount);

    // 求总页码
    Integer pageTotal = pageTotalCount / pageSize;
    if(pageTotalCount % pageSize > 0) {
        pageTotal += 1;
    }

    // 设置总页码
    page.setPageTotal(pageTotal);

    // 求当前页数据的开始索引
    int begin = (page.getPageNo() - 1) * pageSize;

    // 求当前页数据
    List<Student> items = studentDao.queryForPageItems(begin, pageSize);

    // 设置当前页数据
    page.setItems(items);

    return page;
}
```

controller代码:

```
@Controller
public class StudentController {

    private StudentService studentService = new StudentServiceImpl();

    // 查询分页后的首页信息
    @RequestMapping(value = "/studentJump/{id}", method = RequestMethod.GET)
    public String getAllStudent(HttpServletRequest request, @PathVariable("id")
Integer pageNo){
        Page<Student> page = studentService.page(pageNo, Page.PAGE_SIZE);
        List<Student> students = page.getItems();
        request.setAttribute("page",page);
        request.setAttribute("students", students);
        return "forward:/student";
    }

}
```

跳转页面代码:

```
<div class="leftContent">
    <a href="${pageContext.request.contextPath}/studentJump/1">学 生 管 理</a>
    <a href="${pageContext.request.contextPath}/tbClassJump">班 级 管 理</a>
    <a href="${pageContext.request.contextPath}/tbActivityJump">活 动 管 理</a>
    <a href="${pageContext.request.contextPath}/tbResourceJump">资 料 下 载</a>
</div>
```

主页面代码：

```
<c:forEach items="${requestScope.students}" var="student">
    <tr>
        <td>${student.stuid}</td>
        <td>${student.sname}</td>
        <td><h3>${student.gender==0 ? "♀":"♂"}</h3></td>
        <td>${student.phone}</td>
        <td>${student.address}</td>
        <td>${student.clsid}</td>
        <td>${student.STATUS==1 ? "在读" : "开除"}</td>
        <td>
            <a href="${pageContext.request.contextPath}/modifyStudentsJump">修改学生信息</a>
            <input type="button" value="开除学生">
        </td>
    </tr>
</c:forEach>
<tr>
    <td colspan="8">当前页: ${requestScope.page.pageNo}, 总页数: ${requestScope.page.pageTotal}</td>
</tr>
```

首页、上一页、下一页、末页实现

```
<td colspan="2">
    <a href="${pageContext.request.contextPath}/studentJump/1">首页</a>
</td>
<td colspan="2">
    <c:if test="${requestScope.page.pageNo > 1}">
        <a href="${pageContext.request.contextPath}/studentJump/${requestScope.page.pageNo-1}">上一页</a>
    </c:if>
</td>
<td colspan="2">
    <%-- 如果已经 是最后一页，则不显示下一页，末页 --%>
    <c:if test="${requestScope.page.pageNo < requestScope.page.pageTotal}">
        <a href="${pageContext.request.contextPath}/studentJump/${requestScope.page.pageNo+1}">下一页</a>
    </c:if>
</td>
<td>
    <a href="${pageContext.request.contextPath}/studentJump/${requestScope.page.pageTotal}">尾页</a>
</td>
```

添加

jsp页面代码：

```
<td><a href="${pageContext.request.contextPath}/addStudentJump/${requestScope.page.pageNo}">添加学生</a></td>
<td colspan="2">
    <a href="${pageContext.request.contextPath}/studentJump/1">首页</a>
```

跳转到添加学生的页面

当前页码

controller页面代码：

```
// 跳转到添加学生页面
@RequestMapping(value = "/addStudentJump/{pageNo}", method = RequestMethod.GET)
public String addStudentJump(HttpServletRequest request, @PathVariable("pageNo") Integer pageNo){
    request.setAttribute( s: "pageNo", pageNo);
    return "student/addStudent";
}
```

将当前页码存放到请求域中

获取当前页码

添加学生jsp页面:

```
<form action="${pageContext.request.contextPath}/addStudent" method="post">
    <h2>添加学生</h2>
    <table class="teacherAddForm" align="center" border="1" cellpadding="0" cellspacing="0" width="300" height="200"...>
    <input type="submit"> <a href="${pageContext.request.contextPath}/studentJump/${requestScope.pageNo}">退出</a>
</form>
/>
```

提交

跳转到当前页码

完成添加：

```
// 添加学生，返回最后一个页码
@RequestMapping(value = "/addStudent", method = RequestMethod.POST)
public String addStudent(HttpServletRequest request, Student student){
    System.out.println(student);
    Integer pageTotal = studentService.addStudent(student);
    return "forward:/studentJump/" + pageTotal;
}

// 修改学生信息
```

添加学生并且返回尾页

跳转到学生管理尾页

修改

jsp页面代码：

```
<a href="${pageContext.request.contextPath}/modifyStudentsJump/${requestScope.page.pageNo}/${student.stuid}">修改学生信息
>
```

跳转到修改学生页面

获取当前页码

获取当前

controller页面代码：

```
// 跳转到修改学生信息
@RequestMapping(value = "/modifyStudentsJump/{pageNo}/{id}", method = RequestMethod.GET )
public String modifyStudentsJump(HttpServletRequest request, @PathVariable("pageNo") Integer page
    request.setAttribute( s: "pageNo", pageNo);

    Student studentInfo = studentService.findStudentInfo(id);
    request.setAttribute( s: "studentInfo", studentInfo);

    return "student/modifyStudents";
}
```

向request域中存放当前页码

向request域中存放当前id学生的信息

跳转到修改页面

修改学生jsp页面

```
<form action="${pageContext.request.contextPath}/modifyStudent/${requestScope.pageNo}" method="post">
    <input type="hidden" name="stuid" value="${requestScope.studentInfo.stuid}">
    <h2>修改学生信息</h2>
    <table class="teacherAddForm" align="center" border="1" cellpadding="0" cellspacing="0" width="300" heigh
        <tr>
            <th>姓名</th>
            <td><input type="text" name="sname" value="${requestScope.studentInfo.sname}"></td>
        </tr>
        <tr>
            <th>性别</th>
```

返回到当时跳转时候的页面

id不可修改，所以通过隐藏域传递

获取当前请求域的值

完成修改：

```
// 修改学生信息
@RequestMapping(value = "/modifyStudent/{pageNo}", method = RequestMethod.POST)
public String modifyStudent(Student student, @PathVariable("pageNo") Integer pageNo)
    studentService.updateStudent(student); 修改
    System.out.println(student);
    return "forward:/studentJump/" + pageNo;
}
```

返回到之前的页面

查找



通过请求路径判断是那个请求

```
// 查询分页后的首页信息
@RequestMapping(value = {"/studentJump/{pageNo}", "/queryStudents/{pageNo}"}, method = {Reque
public String getAllStudent(HttpServletRequest request, @PathVariable("pageNo") Integer pageNo,
                            @RequestParam(value="clsid", required = false, defaultValue="--全部班级
                            @RequestParam(value="stuid", required = false, defaultValue="")String

    String servletPath = request.getServletPath(); // 获取Servlet的路径
    System.out.println(servletPath);
```

```
<td colspan="2">
    <c:if test="${requestScope.page.pageNo > 1}">
        <a href="${pageContext.request.contextPath}/${sessionScope.servletPath}/${requestScope.p
    </c:if>
    <c:if test="${requestScope.page.pageNo <= 1}">
        <input type="button" value="上一页">
    </c:if>
</td>
<td colspan="2">
    <%-- 如果已经 是最后一页，则不显示下一页，末页 --%>
    <c:if test="${requestScope.page.pageNo < requestScope.page.pageTotal}">
        <a href="${pageContext.request.contextPath}/${sessionScope.servletPath}/${requestScope.p
    </c:if>
    <c:if test="${requestScope.page.pageNo >= requestScope.page.pageTotal}">
        <input type="button" value="下一页">
    </c:if>
```

获取班级和id

```
<form action="${pageContext.request.contextPath}/queryStudents/1" method="get">
    <select id="clsid" name="clsid">
        <option value=""></option>
        <option value="--全部班级--">--全部班级--</option>
        <option value="1903">1903</option>
        <option value="1904">1904</option>
        <option value="1905">1905</option>
        <option value="其他">其他</option>
    </select>
    学号: <input type="text" name="stuid" value="${sessionScope.stuid}">
    <input type="submit" value="查询">
</form>
```

保存到session域中

```
request.getSession().setAttribute( s: "servletPath", o: "studentJump");
request.getSession().setAttribute( s: "clsid", clsid);
request.getSession().setAttribute( s: "stuid", stuid);
System.out.println("过了？？？");
Page<Student> page = studentService.page(pageNo, Page.PAGE_SIZE);
List<Student> students = page.getItems();
request.setAttribute( s: "page",page);
request.setAttribute( s: "students", students);
return "forward:/student";
```

studentJump是默认值，默认访问全部班级的全部学生

queryStudents是通过输入（班级、id）查询

```
@RequestMapping(value = {"/studentJump/{pageNo}", "/queryStudents/{pageNo}"}, method = {RequestMethod.GET, Reque
public String getAllStudent(HttpServletRequest request, @PathVariable("pageNo") Integer pageNo,
                            @RequestParam(value="clsid", required = false, defaultValue="--全部班级--")String clsid,
                            @RequestParam(value="stuid", required = false, defaultValue="")String stuid){
```

通过尾部获得参数，来知道是进行哪一个查询



```
questScope.page.pageTotal}</td>

{requestScope.page.pageNo}">添加学生</a></td>

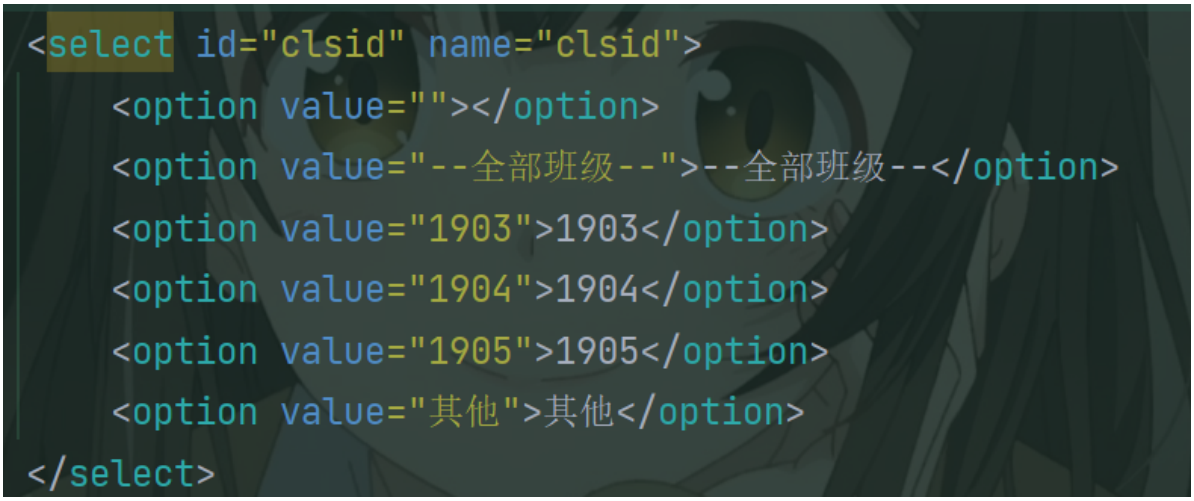
servletPath}/1?clsid=${sessionScope.clsid}&stuid=${sessionScope.stuid}">首页</a>

pe.servletPath}/${requestScope.page.pageNo-1}?clsid=${sessionScope.clsid}&stuid=${sessionScope.stuid
```

```
@RequestParam(value="clsid", required = false, defaultValue="--全部班级--")String clsid,
@RequestParam(value="stuid", required = false, defaultValue="")String stuid){
```

遇到问题1

刷新页面<select>无法获取值



```
<select id="clsid" name="clsid">
    <option value=""></option>
    <option value="--全部班级--">--全部班级--</option>
    <option value="1903">1903</option>
    <option value="1904">1904</option>
    <option value="1905">1905</option>
    <option value="其他">其他</option>
</select>
```

解决办法

```
<script type="text/javascript">
    // onload事件是浏览器解析完页面之后就会自动融发的事件
    window.onload = function ()  {
        var clsid=document.getElementById("clsid");
        var x = "${sessionScope.clsid}";
        for (var i = 0; i < clsid.length; i++) {
            if (clsid[i].value === x) {
                clsid[i].selected=true;
            }
        }

    }
</script>
```

完成查询功能

如果地址是queryStudents的话，“\\d+”表示可以接受任意数字

pageNo == 1时，意思是点击了查询，第一次访问，给班级和id一个初始化

```
if(Pattern.compile("/queryStudents/\\d+").matcher(servletPath).find()) {

    if (pageNo == 1) {
        request.getSession().setAttribute("servletPath","queryStudents");
        request.getSession().setAttribute("clsid", clsid);
        request.getSession().setAttribute("stuid", stuid);
    }

    String clsId = (String) request.getSession().getAttribute("clsid");
    String stuId = (String) request.getSession().getAttribute("stuid");
    System.out.println(clsId);
    System.out.println(stuId);

}
```

删除

添加是否真的要删除

```
<script type="text/javascript" src="static/js/jquery-1.7.2.js"></script>
<script type="text/javascript">
    $(function () {
        // 给删除绑定单击事件
        $("a.deleteItem").click(function () {
            return confirm("你确定要删除吗？")
        });
    });
</script>
```

使用 Filter 过滤器拦截所有内容，实 现权限检查

```
public class teacherFilter implements Filter {
    @Override
    public void doFilter(ServletRequest servletRequest, ServletResponse
servletResponse, FilterChain filterChain) throws IOException, ServletException {
        HttpServletRequest httpServletRequest = (HttpServletRequest)
servletRequest;
        Object teacher =
httpServletRequest.getSession().getAttribute("teacher");
        if (teacher == null) {

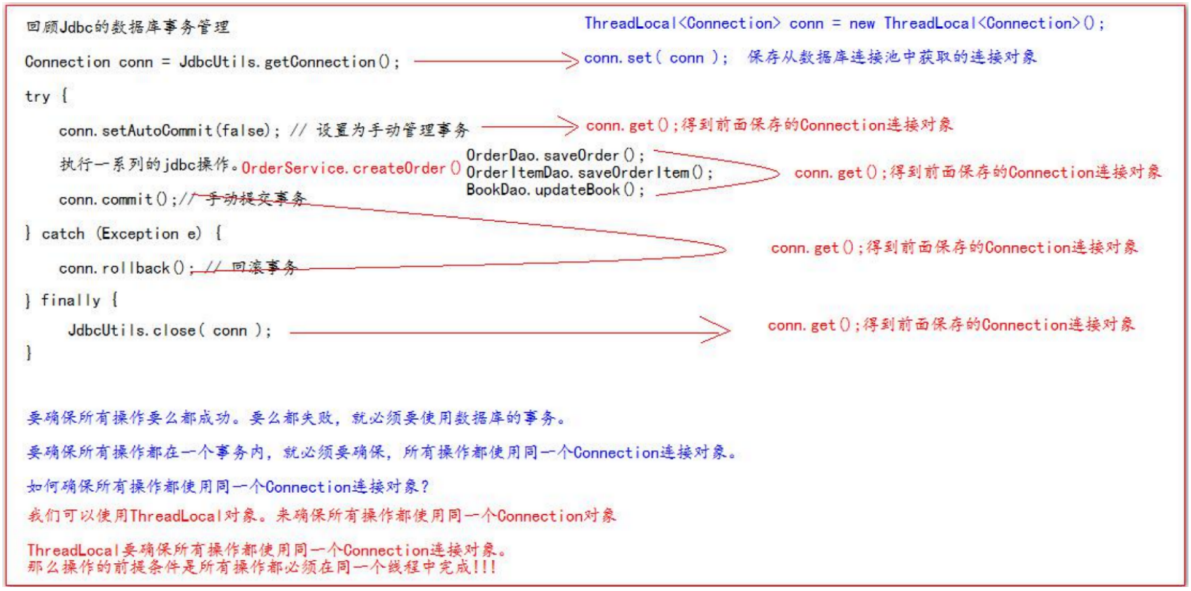
 httpServletRequest.getRequestDispatcher("/").forward(servletRequest,servletResp
onse);
        } else {
            filterChain.doFilter(servletRequest, servletResponse);
        }
    }
}
```

在 web.xml 中的配置:

```
<!--实现权限检查-->
<filter>
   <filter-name>teacherFilter</filter-name>
   <filter-class>com.isjhd.filter.teacherFilter</filter-class>
</filter>
<filter-mapping>
   <filter-name>teacherFilter</filter-name>
   <!-- /* 表示当前工程下所有请求 -->
   <url-pattern>/*</url-pattern>
</filter-mapping>
```

使用 Filter 和 ThreadLocal 组合管理事务

使用 ThreadLocal 来确保所有 dao 操作都在同一个 Connection 连接对象中完成




```
/**
```

```
 * 获取数据库连接池中的连接
 * @return 如果返回 null,说明获取连接失败<br/>有值就是获取连接成功
 */
public static Connection getConnection(){
    Connection conn = conns.get();

    if (conn == null) {
        try {
            conn = dataSource.getConnection();
            conns.set(conn); //保存到 ThreadLocal 对象中，供后面的jdbc操作使用
            conn.setAutoCommit(false); // 设置手动管理事务
        } catch (Exception e) {
            e.printStackTrace();
        }

    }
    return conn;
}
```

提交事务，并关闭释放连接

```
    /**
     * 提交事务，并关闭释放连接
     */
    public static void commitAndClose() {
        Connection connection = conns.get();
        if (connection != null) { // 如果不等于null，说明之前使用过连接，操作过数据库
            try {
                connection.commit(); // 提交事务
            } catch (SQLException e) {
                e.printStackTrace();
            } finally {
                try {
                    connection.close(); // 关闭连接，释放资源
                } catch (SQLException e) {
                    e.printStackTrace();
                }
            }
        }
        // 一定要执remove操作，否则就会出错。(因为Tomcat服务器底层使用了线程池技术)
        conns.remove();
    }
```

回滚事务，并关闭释放连接

```
/**
     * 回滚事务，并关闭释放连接
     */
    public static void rollbackAndClose() {
        Connection connection = conns.get();
        if (connection != null) { // 如果不等于null，说明之前使用过连接，操作过数据库
            try {
                connection.rollback(); // 回滚事务
            } catch (SQLException e) {
                e.printStackTrace();
```

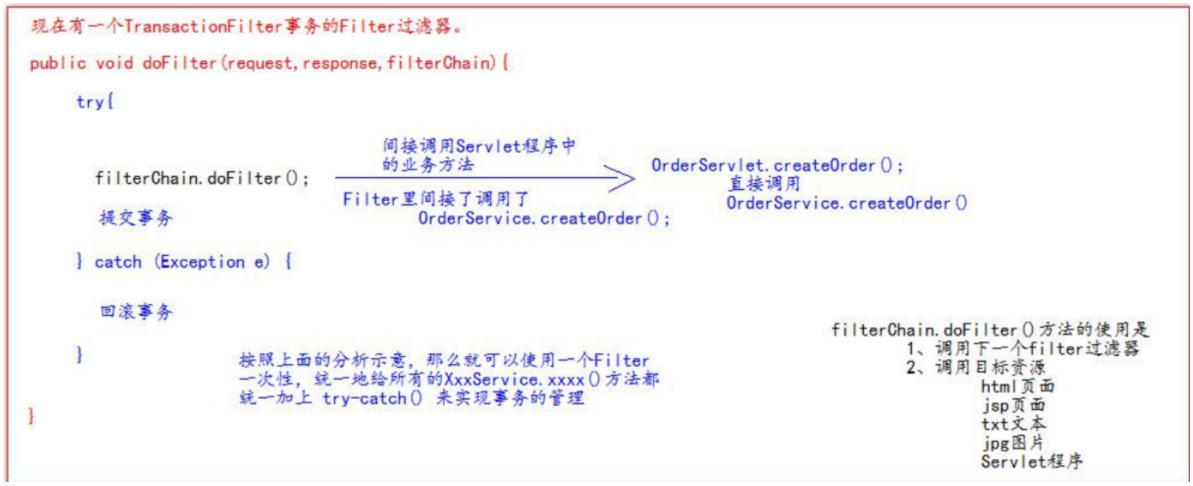
```
            } finally {
                try {
                    connection.close(); // 关闭连接，释放资源
                } catch (SQLException e) {
                    e.printStackTrace();
                }
            }
        }
        // 一定要执remove操作，否则就会出错。(因为Tomcat服务器底层使用了线程池技术)
        conns.remove();
    }
```

修改BaseDao

```
public class BaseDao {
    //使用DbUtils操作数据库
    private QueryRunner queryRunner = new QueryRunner();

    // update() 方法用来执行：Insert\Update\Delete语句
    // return 如果返回-1，说明执行失败，返回其他表示影响到的行数
    public int update(String sql, Object ... args) {
        Connection connection = JdbcUtils.getConnection();
        try {
            return queryRunner.update(connection, sql, args);
        } catch (SQLException e) {
            e.printStackTrace();
            throw new RuntimeException(e);
        }
    }
}
```

使用 Filter 过滤器统一给所有的 Service 方法都加上 try-catch。来进行实现的管理。



Filter 类代码：

```
public class TransactionFilter implements Filter {
```

```
    @Override
    public void doFilter(ServletRequest servletRequest, ServletResponse
servletResponse, FilterChain filterChain) throws IOException, ServletException {
        try {
            filterChain.doFilter(servletRequest, servletResponse);
            JdbcUtils.commitAndClose(); // 提交事务
        } catch (Exception e) {
            JdbcUtils.rollbackAndClose(); // 回退事务
            e.printStackTrace();
            throw new RuntimeException(e);
        }
    }
}
```

在 web.xml 中的配置：

```
<!--使用 Filter 和 ThreadLocal 组合管理事务-->
<filter>
   <filter-name>TransactionFilter</filter-name>
   <filter-class>com.isjhd.filter.TransactionFilter</filter-class>
</filter>
<filter-mapping>
   <filter-name>TransactionFilter</filter-name>
   <!-- /* 表示当前工程下所有请求 -->
   <url-pattern>/*</url-pattern>
</filter-mapping>
```

将所有异常都统一交给 Tomcat，让 Tomcat 展示友好的错误信息页面。

error404.jsp 错误页面：

```
<h1>很抱歉，你访问的页面不存在</h1>
<span style="height: 50px"><img src="static/images/1.gif"/></span>
<h2><a href="${pageContext.request.contextPath}/">返回首页</a></h2>
```

web.xml中配置

```
<!-- error-page 标签配置，服务器出错之后，自动跳转的页面 -->
<error-page>
   <!-- error-code 是错误类型 -->
   <error-code>404</error-code>
   <!-- location 标签表示：要跳转去的页面路径 -->
   <location>/error404</location>
</error-page>
```

文件下载

使用ResponseEntity实现下载文件的功能

```
@RequestMapping("/testDown")
    public ResponseEntity<byte[]> testResponseEntity(HttpSession session) throws
IOException {
        //获取ServletContext对象
        ServletContext servletContext = session.getServletContext();
        //获取服务器中文件的真实路径
        String realPath = servletContext.getRealPath("/static/images/4.png");
        //创建输入流
        InputStream is = new FileInputStream(realPath);
        //创建字节数组
        byte[] bytes = new byte[is.available()];
        //将流读到字节数组中
        is.read(bytes);
        //创建HttpHeaders对象设置响应头信息
        MultiValueMap<String, String> headers = new HttpHeaders();
        //设置要下载方式以及下载文件的名字
        headers.add("Content-Disposition", "attachment;filename=4.png");
        //设置响应状态码
        HttpStatus statusCode = HttpStatus.OK;
        //创建ResponseEntity对象
        ResponseEntity<byte[]> responseEntity = new ResponseEntity<>(bytes,
headers,
                statusCode);
        //关闭输入流
        is.close();
        return responseEntity;
    }
```

文件上传

文件上传要求form表单的请求方式必须为post，并且添加属性enctype="multipart/form-data"
SpringMVC中将上传的文件封装到MultipartFile对象中，通过此对象可以获取文件相关信息

```
<form th:action="@{/testUp}" method="post" enctype="multipart/form-data">
    头像：<input type="file" name="photo"><br>          必须填写
    <input type="submit" value="上传">
</form>
```

a>添加依赖：

```
<dependency>
    <groupId>commons-fileupload</groupId>
    <artifactId>commons-fileupload</artifactId>
    <version>1.3.1</version>
</dependency>
```

b>在SpringMVC的配置文件中添加配置：

```
<!--必须通过文件解析器的解析才能将文件转换为MultipartFile对象-->
<bean id="multipartResolver"
class="org.springframework.web.multipart.commons.CommonsMultipartResolver">
</bean>
```

c>控制器方法：

```
    @RequestMapping("/testUp")
    public String testUp(MultipartFile photo, HttpSession session) throws
IOException {
        // 获取上传的文件的文件名
        String fileName = photo.getOriginalFilename();
        // 处理文件重名问题
        String hzName = fileName.substring(fileName.lastIndexOf("."));
        fileName = UUID.randomUUID().toString() + hzName;
        // 获取服务器中photo目录的路径
        ServletContext servletContext = session.getServletContext();
        String photoPath = servletContext.getRealPath("static\\images");
        File file = new File(photoPath);
        if (!file.exists()) {
            file.mkdir();
        }
        String finalPath = photoPath + File.separator + fileName;
        // 实现上传功能
        photo.transferTo(new File(finalPath));
        return "success";
    }
```